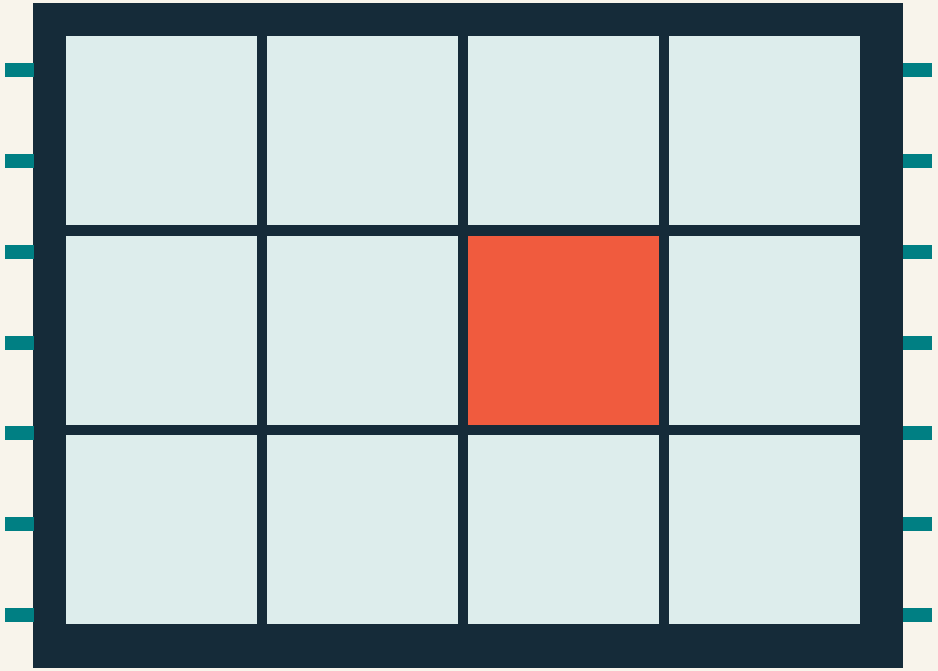
# Make your own chip.

From an 8-bit counter
to a real ASIC.

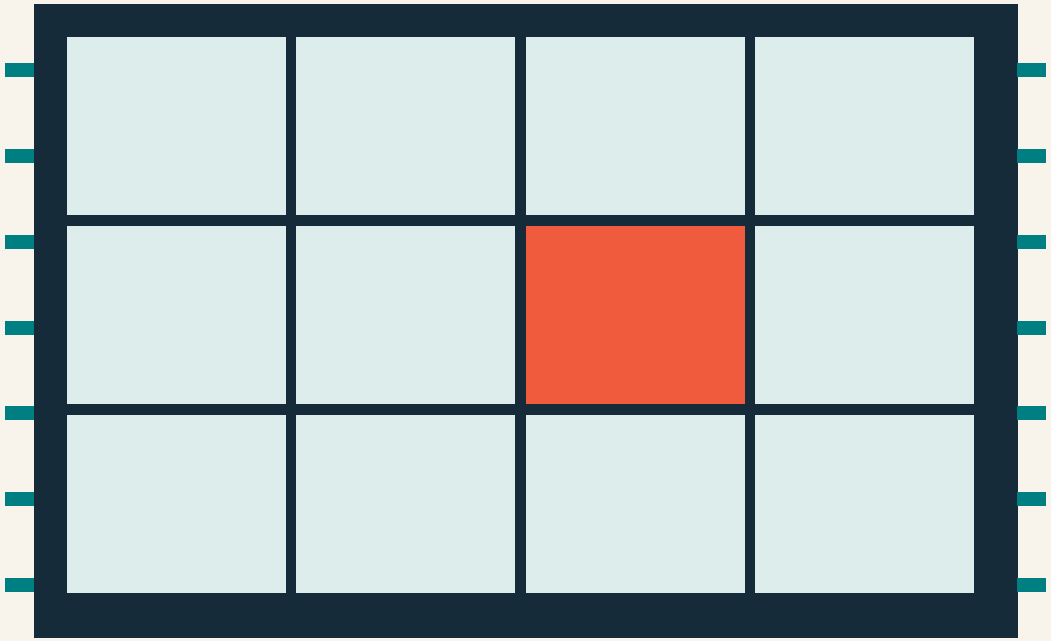
ASIC = a chip made for a specific job.
Our job: count from 0 to 255.



ONE SHARED CHIP. YOUR OWN TILE.



# Many designs. One shared chip.



Select one tile

Your board pins

**Shared fabrication**

**Open design tools**

**Open PDK: process rules + cells**

# The path from an idea to silicon

1 → 2 → 3 → 4

## Idea
Count 0–255

## RTL
Verilog: registers + logic

## Test
Check its behaviour

## Template
Connect fixed pins

5 → 6 → 7 → 8

## Actions
Tools build + check

## GDS
Layout geometry

## Fab
Manufacture

## Board
Select + test

YOU DESIGN AND VERIFY • ACTIONS BUILD • FABRICATION FOLLOWS AN APPROVED SUBMISSION

# Choose the process with your template

## IHP

**130** nm

SG13G2
IHP MPW

## SkyWater

**130** nm

SKY130
ChipFoundry

## GlobalFoundries

**180** nm

GF180
wafer.space

**Allow months for fabrication, assembly and delivery: typically 6–12 months.**

# Eight bits become eight output pins

ui_in[0] → enable
clk → clock
rst_n = 0 → reset




uo_out[7:0] → count
uio[0] → wrap
uio_oe[0] = 1

**EXAMPLE: COUNT = 5 → BINARY 00000101**

| uo[7] | uo[6] | uo[5] | uo[4] | uo[3] | uo[2] | uo[1] | uo[0] |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |

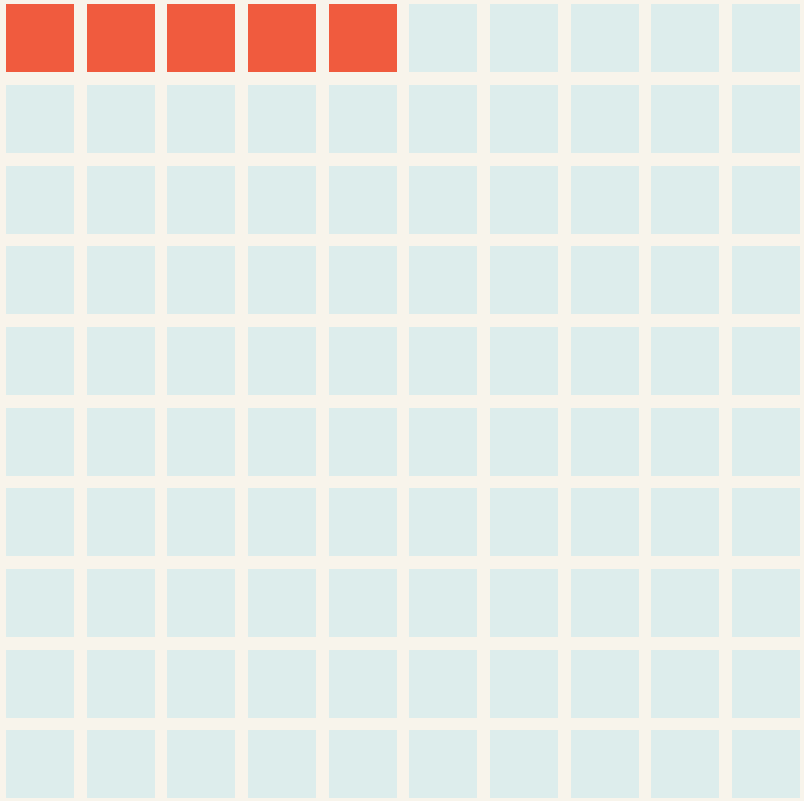
**1x1: SKY / IHP ≈ 167 × 108 µm • GF180 ≈ 340 × 160 µm**

# Our counter: 4.96% reported core utilisation

**4.96%**

Utilisation = occupied cell area
as a fraction of available core area.

| | |
|---|---|
| **47** | cells after synthesis |
| **50 MHz** | clock target |
| **+8.643 ns** | worst setup slack |



**SKY130 • 1x1 • commit 61ef06b**

# Draw gates, or write their behaviour

WOKWI • GATES IN A BROWSER

## A picture of the circuit




VERILOG • THIS LESSON

## A description of the circuit

```
assign y = a & b;
```

An IHP Chisel template also exists.

# 01 Copy the template for your class shuttle

| | | |
|---|---|---|
| **SKY130** | ttsky-verilog-template | ttsky26d |
| **IHP** | ttihp-verilog-template | ttihp26b • closed |
| **GF180** | ttgf-verilog-template | ttgf26c |

**P1 • CAPTURE PENDING**

## Use this template

Signed-in template menu capture.

**P2 • CAPTURE PENDING**

## Pages → GitHub Actions

Capture the repo Pages setting.

# 02 Know which file you are changing

```
src/
  project.v
  counter.v
info.yaml
test/
  test.py
  tb.v
  Makefile
docs/info.md
```

## Circuit

src/: wrapper + counter RTL

## Build description

info.yaml: name, sources, clock, tiles

## Tests

test/: stimulus, checks and instance

## Datasheet

docs/info.md: behaviour + testing

# 03 Connect the counter to the shared pins

## The port contract

8 inputs: ui_in

8 outputs: uo_out

8 bidirectional: uio_*

Control: ena, clk, rst_n

**uio_oe: 0 = input**
**uio_oe: 1 = output**

```
counter #(.WIDTH(8)) counter_inst (
  .clk(clk), .rst(~rst_n),
  .en(ui_in[0]),
  .count(uo_out),
  .wrap(uio_out[0])
);
assign uio_out[7:1] = 7'b0;
assign uio_oe = 8'b0000_0001;
```

**Same top name in RTL, info.yaml and tb.v.**

# 04 Describe what happens on a clock edge

```
always @(posedge clk) begin
  if (rst) count <= 8'b0;
  else if (en) count <= count + 1'b1;
end
assign wrap = en && (&count);
```

rst_n = 0 → 0 at next ↑clk

Enable = 1 → increment

Enable = 0 → hold

wrap = 1 at 255, if enabled




**Try it: count = 7, enable = 0. Next count? 7 — it holds.**

# 05 A test says what should happen

## Excerpt using step() from the real test

```
await step(reset=True)
await step(enable=1)
assert int(dut.uo_out.value) == 1
await step(enable=0)
assert int(dut.uo_out.value) == 1
```

Needs Python + make + Icarus Verilog

```
cd test && make -B
python -m cocotb_tools.check_results results.xml
```

**RESET → 0**

**COUNT → 1**

**HOLD → 1**

step() applies inputs and samples after the edge.

# 06 Push once. Check three green workflows.

A commit starts three automated jobs: test checks behaviour, gds builds and checks layout, docs writes the datasheet.




**docs PASS**

Datasheet generated

**test PASS**

Reset, count, hold, wrap

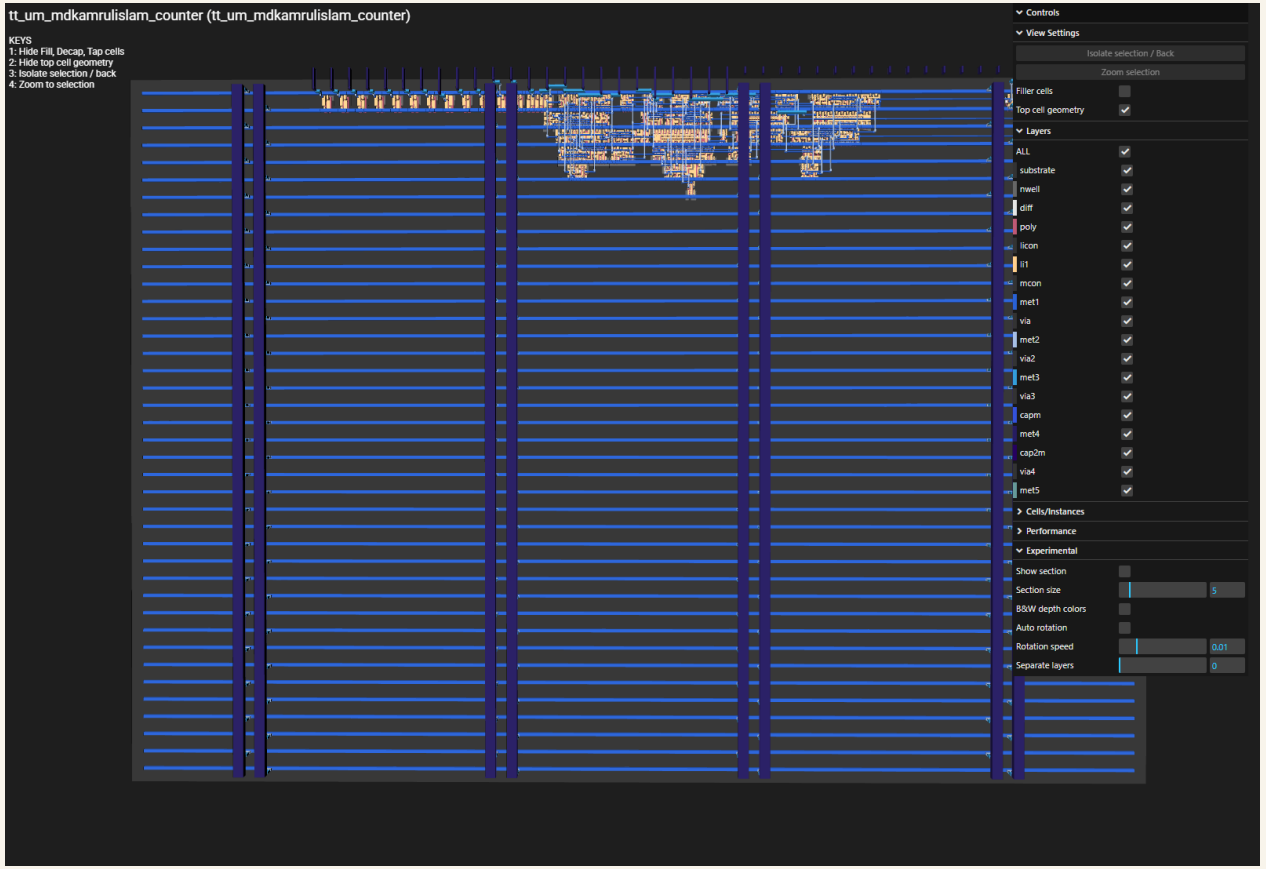
**gds PASS**

All four jobs succeeded

## Worked example: tt-sky130-counter • commit 61ef06b

ACTUAL GITHUB EVIDENCE • FPGA IS OPTIONAL AND WAS NOT RUN

# 07 A layout is geometry, not a photograph




## Look for three things

1 Logic + wiring

2 Warnings + timing

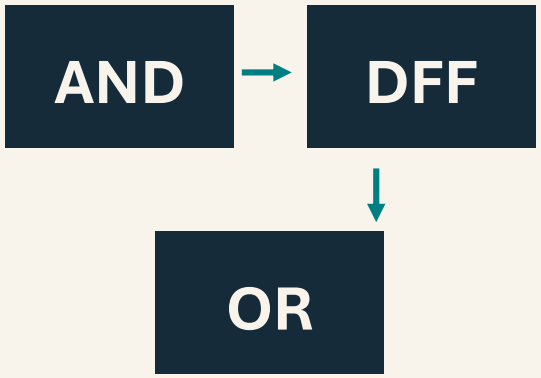
3 Precheck + gl_test

**Open the actual viewer ↗**

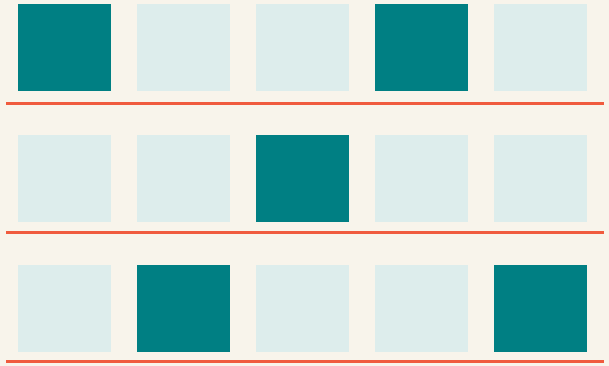
Filler cells hidden.
P4 metrics capture pending.

# Tools turn behaviour into cells and wires

## Map logic




## Place + route



## Check

- Timing
  Before next edge?
- DRC
  Wire spacing okay?
- LVS
  Same connections?

**Yosys + ABC**

**OpenROAD**

**Timing + DRC + LVS**

# 08 Check the synthesised circuit too

**One testbench**
**Same outputs**

## RTL: your Verilog description

0 → 1 → hold 1 → reset 0

## Gates: the mapped cell netlist

0 → 1 → hold 1 → reset 0

**In test/: make -B GATES=yes • needs gate_level_netlist.v and PDK_ROOT.**

# A real first failure: empty project information

BEFORE • TEMPLATE DEFAULTS

```
title: ""
author: ""
description: ""
```

AFTER • DESCRIBE THE COUNTER

```
title: "8-bit counter"
author: "Your name"
description: "Count, hold, reset"
```

YAML excerpts: these fields belong inside project:

**Fix one cause → commit → rerun → read the new result.**

Then check: top name • source files • assertions • tile area • timing.
No counter assertion failed; this genuine example is a metadata error.

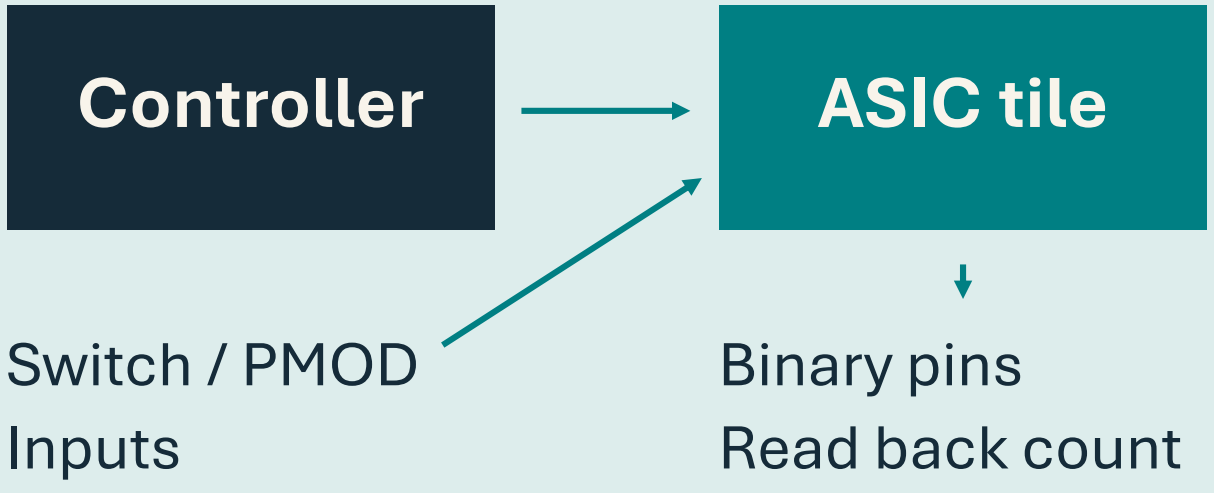
# 09 Choose the revision for fabrication




| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| **Sign in** | **Choose shuttle** | **Add repo** | **Review price** | **Submit revision** |
| GitHub account | Correct fab | Public source | Payment / coupon | Verified commit |

**A new GitHub push needs a new submitted revision before the deadline.**

# On the board, test the same behaviour




No seven-segment decoder is included.

Select your project.

Set a slow clock.

rst_n = 0 until ↑clk:

count becomes 0.

Enable: 0 → 1 → 2.

Disable: count holds.

**Synchronous enable assumed: synchronise and debounce a real switch.**

# Show your evidence before a submission review

- Behaviour and pins are written down.
- RTL, metadata and test setup agree.
- Reset, count, hold and wrap tests pass.
- test, gds and docs pass at one commit.
- Area, timing, warnings and datasheet are reviewed.
- Teacher approves shuttle, terms, budget and revision.

# Lab: make the counter count down

EXPECTED COUNT-DOWN SEQUENCE

0 → 255 → 254

```
count <= count - 1'b1;
```

1 Define the new wrap / underflow rule.

2 Change RTL + expected test values.

3 Push; compare area and timing.

**Unsigned 8-bit count: what is 0 – 1?**
**255 (it wraps).**

**Submit a commit with evidence; no shuttle booking during this exercise.**

# Learn one layer deeper

| | |
|---|---|
| **Start** | Templates ↗ |
| **Connect** | HDL interface rules ↗ |
| **Verify** | Testing guide ↗ |
| **Understand** | Making ASICs + factory tour ↗ |
| **Check limits** | Technical specs ↗ |
| **Plan** | Shuttles + FAQ ↗ |

# Staff: budget projects and shared hardware

**5**

projects, up to

1 PCB kit

**€565**

**25**

projects, up to

3 PCB kits

**€2,195**

**75**

projects, up to

5 PCB kits

**€5,325**

**75 one-tile projects can cover a class of about 70; five kits are shared.**

# Staff: a template pin is not an open booking

| TEMPLATE | ACTION PIN | SUBMISSION DEADLINE |
|---|---|---|
| **SKY** | ttsky26d | 30 Nov 2026 |
| **GF** | ttgf26c | 7 Dec 2026 |
| **IHP** | ttihp26b | Closed 21 Sep 2026 |

**Next IHP: March 2027 planned; exact day unpublished.**
**Estimated shipping: Q1 2028. Re-check before committing.**

# Staff: verify the exact IHP loan agreement

### REPORTED IN PRIOR PROGRAMME MATERIAL

IHP retains chip ownership.

Default loan: two years.

No resale or transfer.

EU + Switzerland shipping.

### CONFIRM FOR THE CHOSEN SHUTTLE

Read the current agreement.

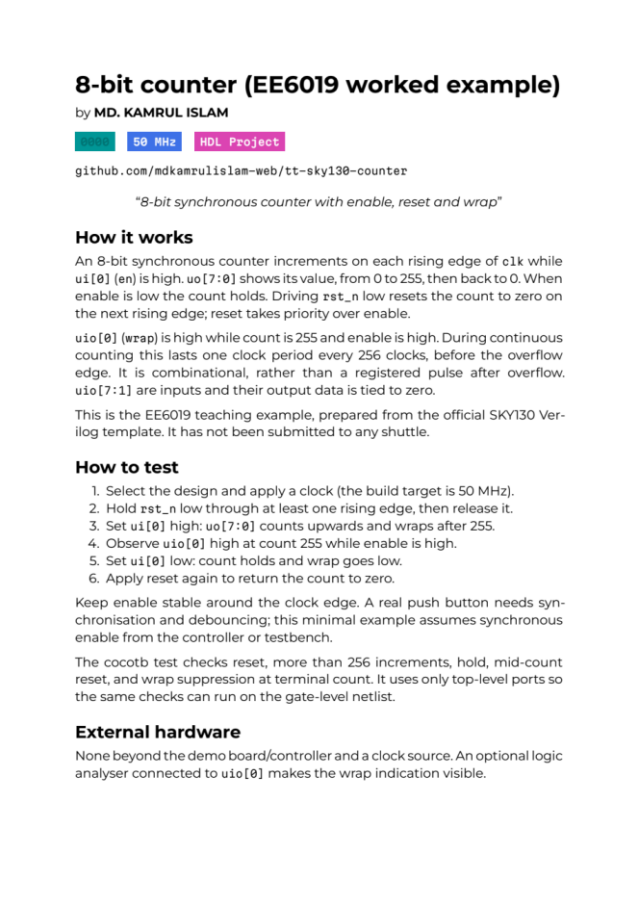
Check student ownership.

Check duration and returns.

Record terms before payment.

**These historical restrictions need official confirmation for the selected run.**

# Staff: finish the evidence pack



## 8-bit counter (EE6019 worked example)

by **MD. KAMRUL ISLAM**

0000 | 50 MHz | HDL Project

github.com/mdkamrulislam-web/tt-sky130-counter

*"8-bit synchronous counter with enable, reset and wrap"*

### How it works

An 8-bit synchronous counter increments on each rising edge of `clk` while `ui[0]` (en) is high. `uo[7:0]` shows its value, from 0 to 255, then back to 0. When enable is low the count holds. Driving `rst_n` low resets the count to zero on the next rising edge; reset takes priority over enable.

`uio[0]` (wrap) is high while count is 255 and enable is high. During continuous counting this lasts one clock period every 256 clocks, before the overflow edge. It is combinational, rather than a registered pulse after overflow. `uio[7:1]` are inputs and their output data is tied to zero.

This is the EE6019 teaching example, prepared from the official SKY130 Verilog template. It has not been submitted to any shuttle.

### How to test

1. Select the design and apply a clock (the build target is 50 MHz).
2. Hold `rst_n` low through at least one rising edge, then release it.
3. Set `ui[0]` high: `uo[7:0]` counts upwards and wraps after 255.
4. Observe `uio[0]` high at count 255 while enable is high.
5. Set `ui[0]` low: count holds and wrap goes low.
6. Apply reset again to return the count to zero.

Keep enable stable around the clock edge. A real push button needs synchronisation and debouncing; this minimal example assumes synchronous enable from the controller or testbench.

The cocotb test checks reset, more than 256 increments, hold, mid-count reset, and wrap suppression at terminal count. It uses only top-level ports so the same checks can run on the gate-level netlist.

### External hardware

None beyond the demo board/controller and a clock source. An optional logic analyser connected to `uio[0]` makes the wrap indication visible.

**Captured: Actions, datasheet, GDS render and real 3D viewer.**

P1 Signed-in template menu

P2 Pages source setting

P4 gds metrics + cell table

P8 Reset / count / hold / wrap waveform

P6 Optional future assertion-failure example

Capture instructions: docs/slides/README.md